بسم الله الرحمن الرحيم

Institut Supérieur d'Informatique de Mahdia

ZAWEDNI

Projet de fin d'études

Conception et déploiement d'une plateforme web et mobile pour l'achat local intégrant une approche DevOps

Réalisé par
Ali Zribi

Formation
Master Professionnel Computer Engineering : Expert Réseaux

Encadrant académique
Amel Zidi

Co-encadrant académique
Ghannay Sana

Encadrant professionnel
Hadil Ben Masseoud

Année universitaire 2025–2026

Plan de la présentation

Sommaire

01 **Contexte et objectifs**
ZAWEDNI, ses utilisateurs et ce que le projet doit apporter

02 **Développement de la plateforme**
Web, mobile, API, service d'IA et base de données

03 **Architecture et démarche DevSecOps**
Le cluster, le réseau, les secrets et les outils

04 **Notre chaîne CI/CD et déploiement**
Du changement de code jusqu'à staging

05 **Supervision et validation**
Métriques, journaux, alertes et preuves

06 **Bilan et perspectives**
Ce qui est réalisé, ce qui reste à faire

Chapitre 1

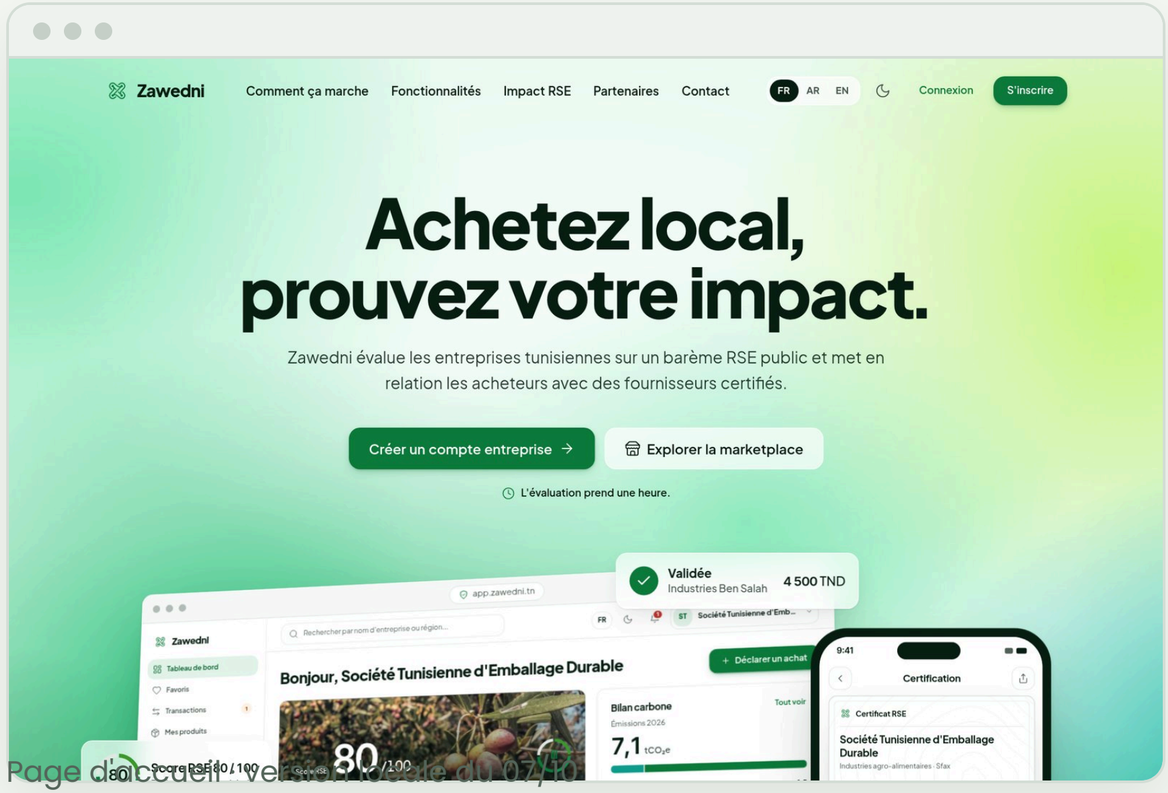
01

Contexte et objectifs

Le besoin, les utilisateurs et la démarche de travail.

01 Contexte et objectifs
02 Développement de la plateforme
03 Architecture et DevSecOps
04 Chaîne CI/CD et déploiement
05 Supervision et validation
06 Bilan et perspectives

ZAWEDNI et l'achat local responsable



Page d'accueil · Version locale du 07/10

1. Un acheteur ne sait pas quels fournisseurs locaux sont vraiment responsables.

2. La recherche de fournisseurs et les demandes de devis se font à la main, par e-mail.

3. Une PME ne sait pas comment améliorer sa démarche RSE.

Une grille RSE publique, un certificat vérifiable et une marketplace de fournisseurs certifiés.

Contexte et objectifs · Utilisateurs

Trois profils d'utilisateurs

Fournisseur

PME, artisan, producteur local

- remplit sa grille RSE
- reçoit son certificat
- publie ses produits
- répond aux demandes de devis

Acheteur

hôtel, entreprise, responsable achats

- cherche des fournisseurs certifiés
- envoie une demande de devis
- déclare ses achats locaux
- suit son bilan carbone

Administrateur

équipe ZAWEDNI

- valide les inscriptions
- examine les grilles RSE
- délivre les certificats
- gère secteurs, régions, questions

Une même entreprise peut être à la fois acheteuse et fournisseuse.

Besoins fonctionnels et exigences de qualité

Fonctionnel — Ce que l'utilisateur fait

Évaluer
remplir la grille RSE, obtenir un score et un palier

Certifier
valider la grille, délivrer un certificat vérifiable par QR code

Trouver et acheter
filtrer la marketplace, demander un devis, déclarer un achat

Livrer
tester, analyser et déployer chaque changement de code

Qualité — Comment le système doit se comporter

Sécurité
rôles, mots de passe chiffrés, secrets jamais en clair, analyses

Traçabilité
savoir quelle version du code tourne

Observabilité
voir les métriques, les journaux, être alerté

Coût maîtrisé
tenir dans le crédit étudiant Azure de 100 $

Démarche de travail

chaîne ajoutée, développement déjà bien avancé (≈ 85 %)

1
Développement
web, mobile, API, service d'IA, base de données

2
Intégration de la chaîne
conteneurs, infrastructure, pipelines, contrôles

3
Déploiement
cluster AKS, environnement staging, GitOps

4
Validation continue
contrôles, supervision, preuves datées

Le développement continue : chaque nouveau changement passe par la chaîne

Démarche itérative inspirée de Scrum : des étapes courtes, chacune revue avant la suivante. Le chiffre de 85 % est une estimation, pas une mesure.

Chapitre 2

02

Développement de la plateforme

Les interfaces, l'API, le service d'IA et les données.

01 Contexte et objectifs

02 Développement de la plateforme

03 Architecture et DevSecOps

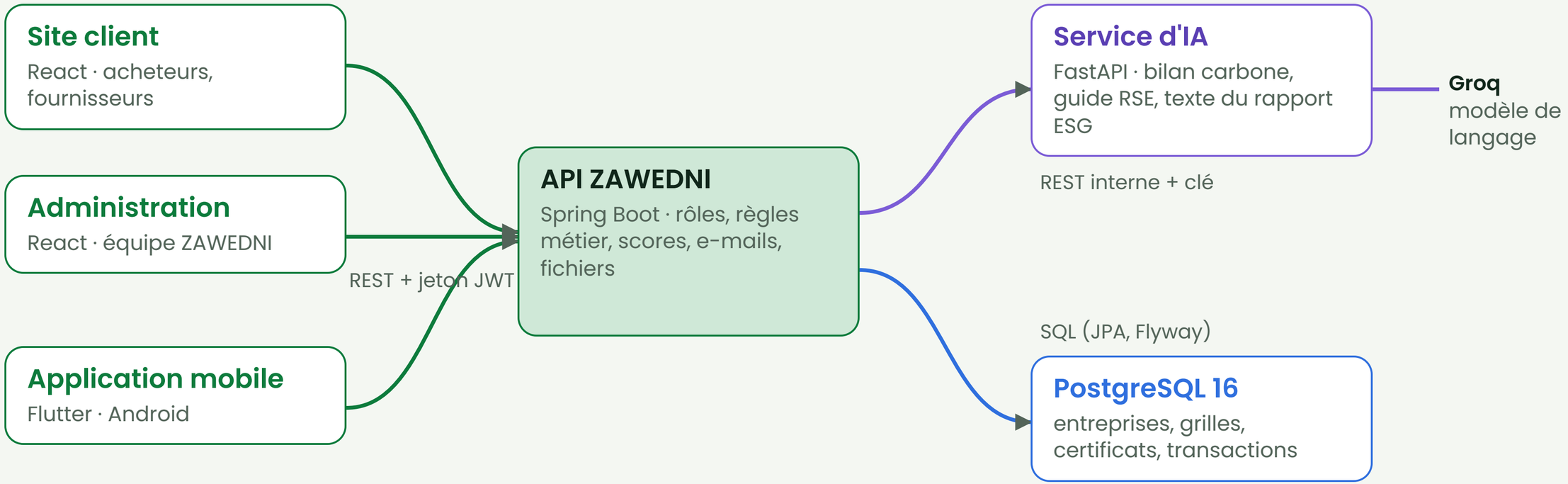
04 Chaîne CI/CD et déploiement

05 Supervision et validation

06 Bilan et perspectives

Développement de la plateforme · Architecture applicative

Architecture de la plateforme



Seule l'API appelle le service d'IA. Les chiffres du rapport ESG viennent de la base ; l'IA rédige seulement le texte.

Développement de la plateforme · Parcours utilisateur

Du fournisseur évalué à l'acheteur

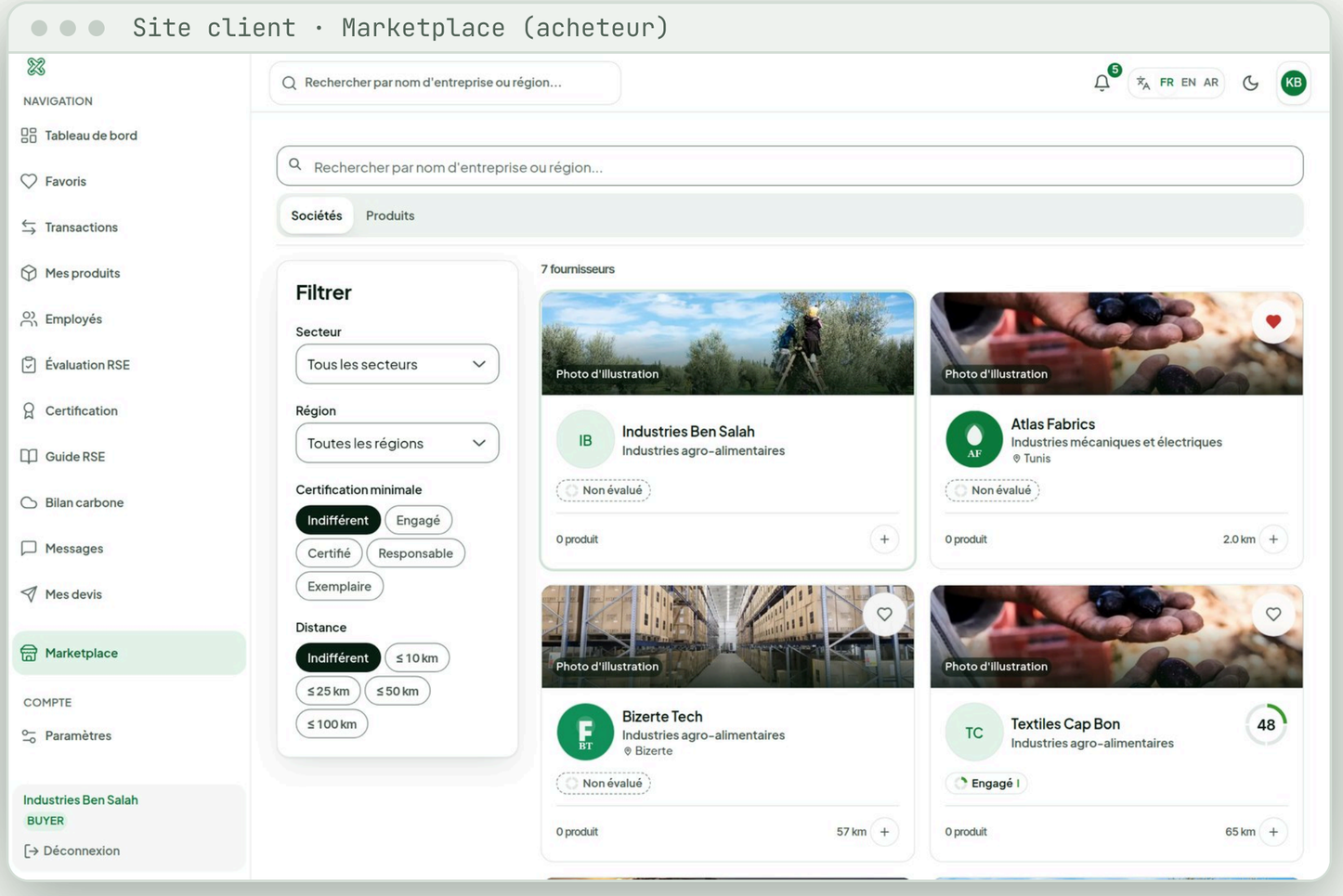
1 Évaluation
le fournisseur remplit sa grille RSE

2 Revue
l'administrateur examine et valide

3 Certificat
score, palier, QR code public

4 Marketplace
l'acheteur le trouve et le contacte

Version locale du 07/10, données de démonstration

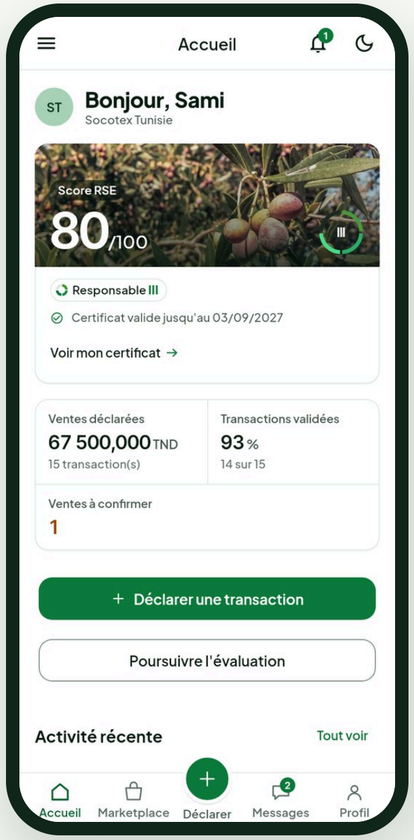


Développement de la plateforme · Mobile

L'application mobile

Même compte, même API. Sur le terrain, le fournisseur suit son score, remplit sa grille et suit ses transactions. L'acheteur parcourt la marketplace.

Écrans Flutter du 06/10, données de démonstration



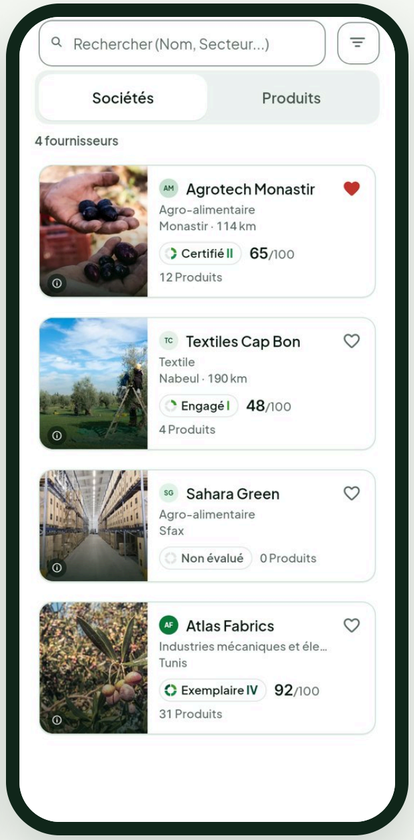
Accueil et score



Grille RSE



Suivi de la revue



Marketplace

Développement de la plateforme · Fonctions

Fonctions métier réalisées

Grille RSE
questions par pilier, brouillon, score calculé

web · mobile · admin

Certificat
palier selon le score, page publique et QR code

web · mobile · admin

Marketplace
entreprises et produits, filtres, favoris

web · mobile

Demandes de devis
envoi, réponses, choix d'une offre

web · mobile

Achats locaux
transaction déclarée puis confirmée, score d'achat local

web · mobile · admin

Bilan carbone
scopes 1, 2 et 3 calculés par le service d'IA

web · mobile

Rapport ESG, guide RSE
chiffres de la base, texte rédigé par l'IA

web · mobile

Échanges et gestion
messages, notifications, inscriptions, référentiels

web · mobile · admin

Développement de la plateforme · Outils

Outils de développement

Outil	Tâche dans ZAWEDNI	Raison du choix
React + Vite	site client et administration	composants réutilisables, même langage pour les deux sites, build rapide
Flutter	application mobile Android	un seul code pour l'interface, rendu fidèle à la maquette
Spring Boot Java 17	API, sécurité, règles métier	sécurité, accès aux données, e-mails et métriques déjà intégrés
FastAPI Python	service d'IA	Python pour les calculs et l'appel au modèle ; API légère et typée
Groq	rédiger le guide RSE et le texte ESG	modèle de langage appelé par API : aucun modèle à héberger
PostgreSQL 16	données de la plateforme	données liées entre elles, transactions fiables ; Flyway versionne le schéma

Chapitre 3

03

Architecture et démarche DevSecOps

Où tournent les services, et comment on les protège.

01 Contexte et objectifs

02 Développement de la plateforme

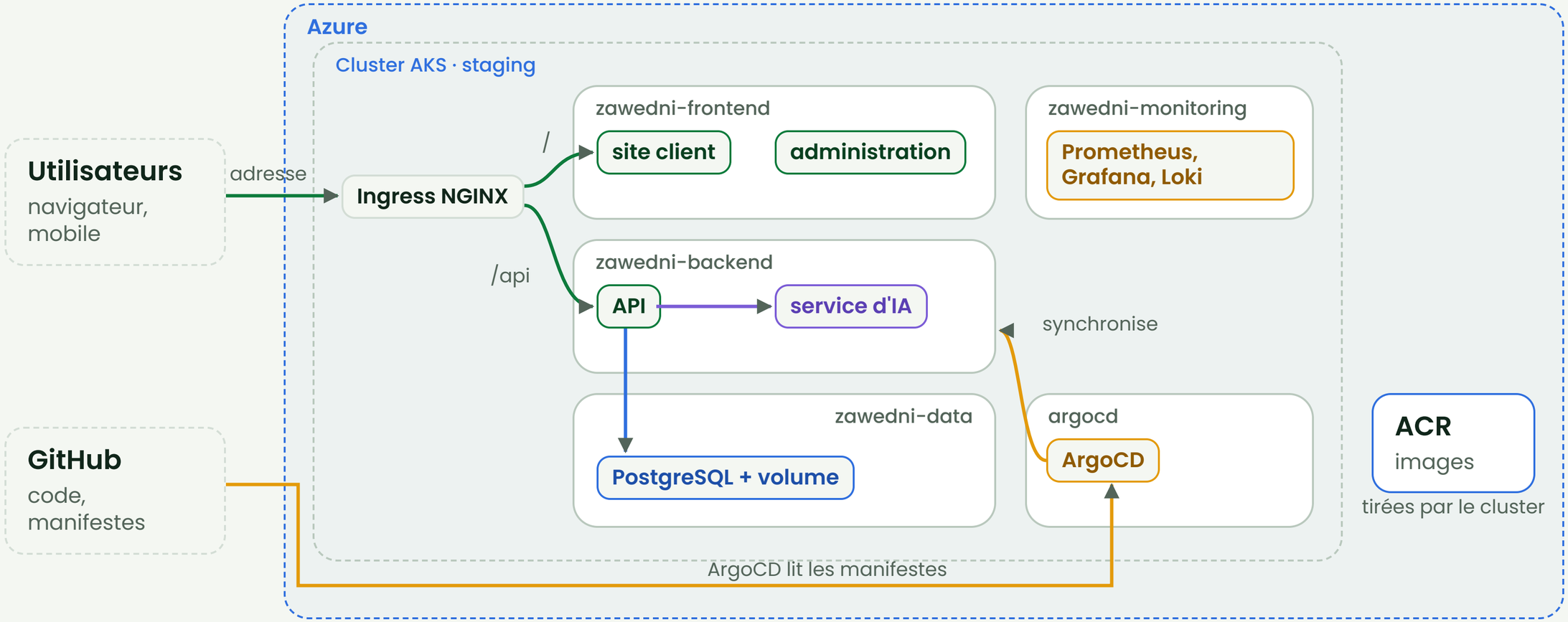
03 Architecture et DevSecOps

04 Chaîne CI/CD et déploiement

05 Supervision et validation

06 Bilan et perspectives

Architecture de déploiement



Staging déployé et testé. La production est décrite dans Git, dans ses propres namespaces, mais n'est pas déployée.

Le cycle DevSecOps

Dans ZAWEDNI : une vérification à chaque étape, à chaque changement



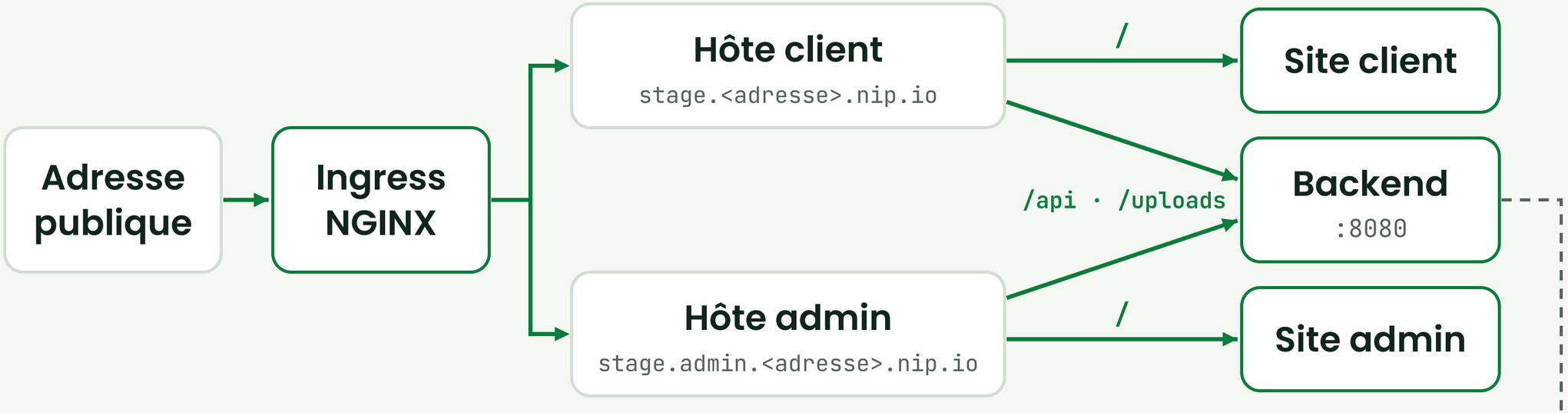
Outils d'infrastructure

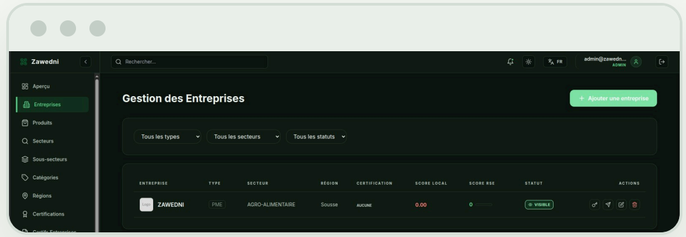
Outil	Rôle	Raison du choix
Docker	une image par service : API, IA, site client, administration	le service tourne de la même façon en local et dans le cluster
Terraform	prépare le cluster, le registre et le stockage	infrastructure écrite en code : on relit le plan, on peut la recréer
AKS	héberge et relance les conteneurs	Kubernetes géré par Azure, payé par le crédit étudiant
ACR	stocke les images versionnées	registre privé dans le même cloud, lu par le cluster sans mot de passe

Outils de livraison, de contrôle et de supervision

Outil	Rôle	Raison du choix
GitHub Actions	lance tests et analyses à chaque pull request ou push	intégré au dépôt, aucun serveur à entretenir
ArgoCD	synchronise le cluster avec Git	l'état déployé est visible et se corrige depuis Git
Contrôles	Gitleaks, SonarCloud, Semgrep, Dependency-Check, Trivy, Checkov	un outil par risque : secrets, code, bibliothèques, images, infrastructure
Secrets	secrets GitHub, Sealed Secrets	aucun mot de passe en clair dans Git
Supervision	Prometheus, Grafana, Alloy, Loki, Alertmanager	outils standards de Kubernetes, installés dans le cluster

Routage des requêtes



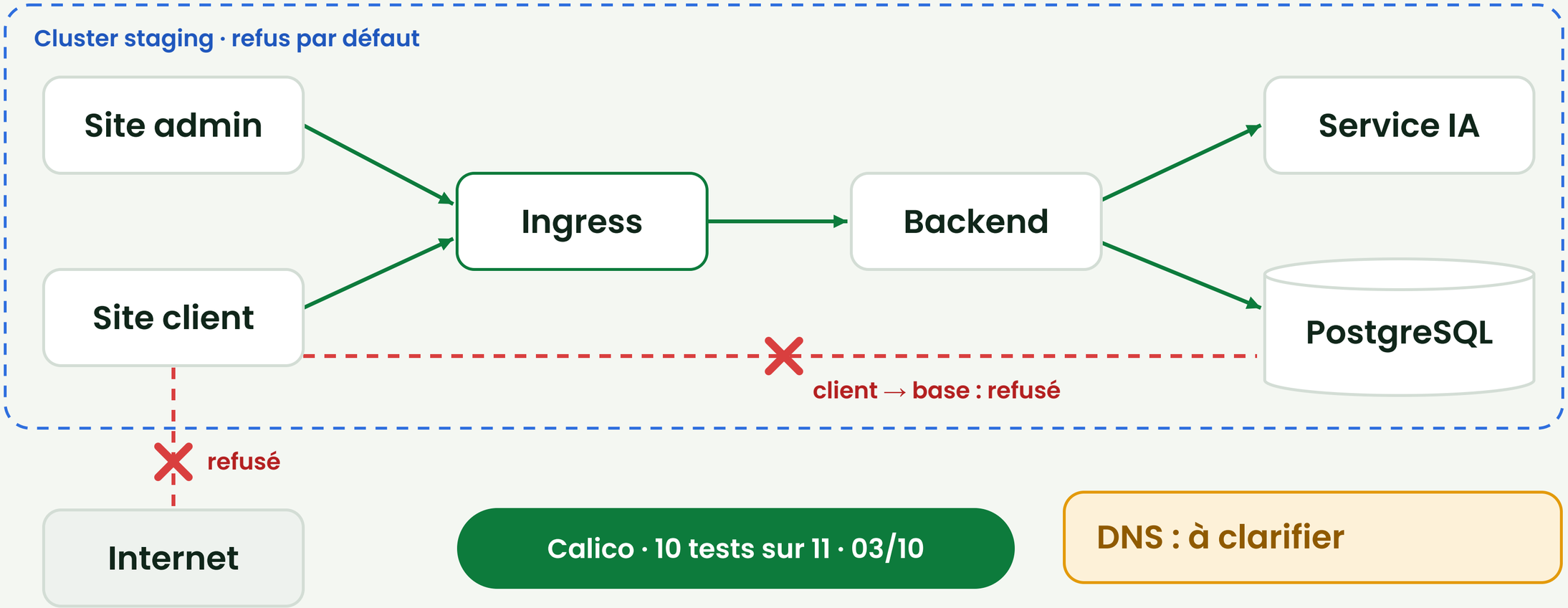


Capture du 06/10 · admin staging

`PUBLIC_HOST` · source unique
vérifiée par ci-public-host

interne, appelé par le backend

Isolation réseau



Secrets et données

Un secret : un mot de passe, une clé ou un token.

Pour le pipeline

Secrets GitHub
`ACR_*, SONAR_TOKEN…`

→

Pipeline
valeur masquée : ***

Pour l'application

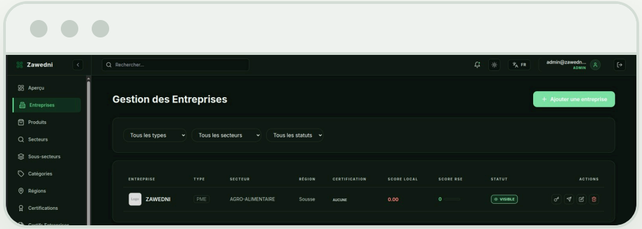
Dépôt Git
Sealed Secret
chiffré

Cluster

déchiffré seulement ici

→ variable `ADMIN_DEFAULT_PASSWORD` →

connexion admin



Capture du 06/10 · admin staging connecté

Données : PostgreSQL et fichiers envoyés sur deux volumes persistants de 5 Go. Un volume n'est pas une sauvegarde : elle reste à automatiser.

Chapitre 4

04

Notre chaîne CI/CD et déploiement

Un changement de code, des contrôles jusqu'à staging.

01 Contexte et objectifs

02 Développement de la plateforme

03 Architecture et DevSecOps

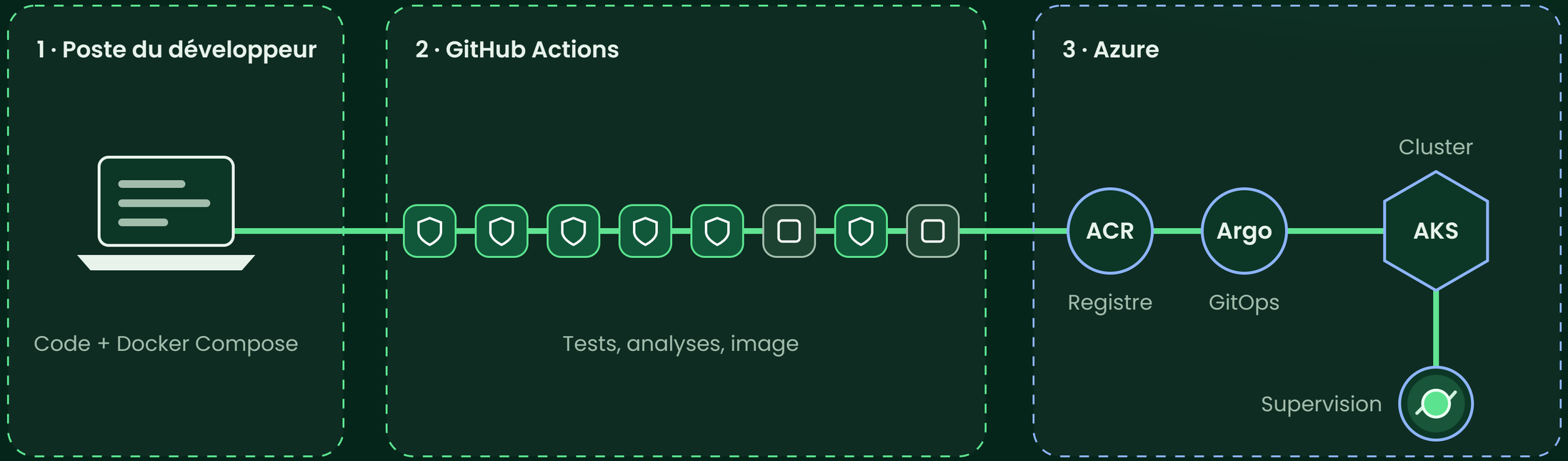
04 Chaîne CI/CD et déploiement

05 Supervision et validation

06 Bilan et perspectives

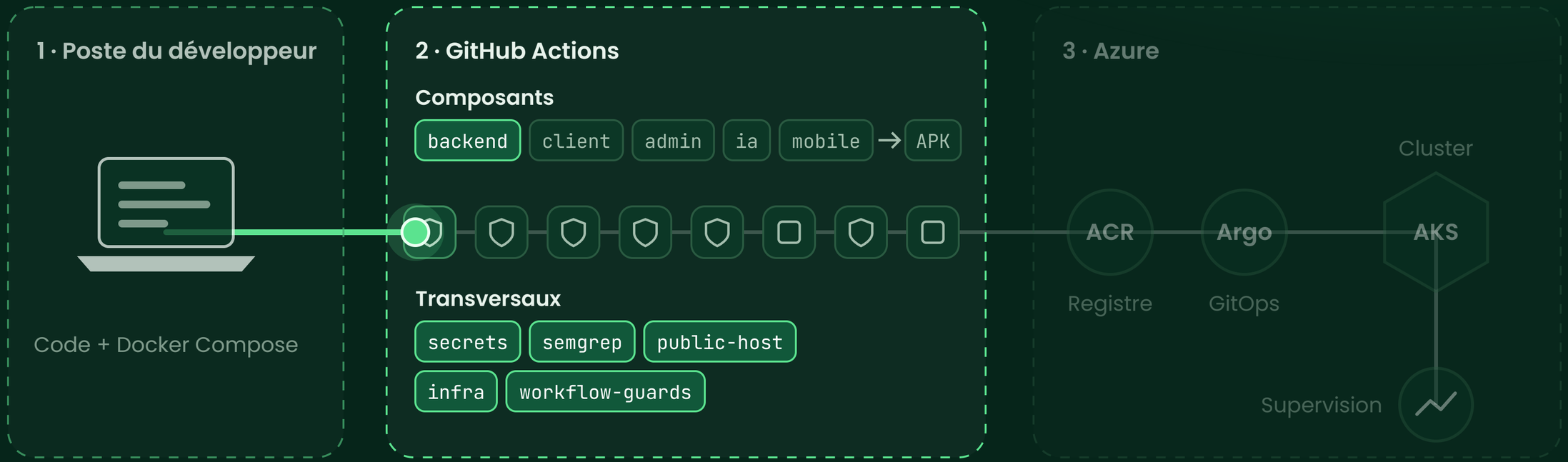
Notre chaîne CI/CD · Vue d'ensemble

Notre chaîne de livraison



Notre chaîne CI/CD · Pipelines

Dix pipelines



Secrets, Semgrep, adresse publique : à chaque pull request et push sur staging ou main.
Infrastructure et workflows : quand leurs fichiers changent.

Notre chaîne CI/CD · Déclencheurs

Ce que déclenche chaque événement

Pull request

- Contrôles des fichiers modifiés
- Si un composant change : image construite et scannée

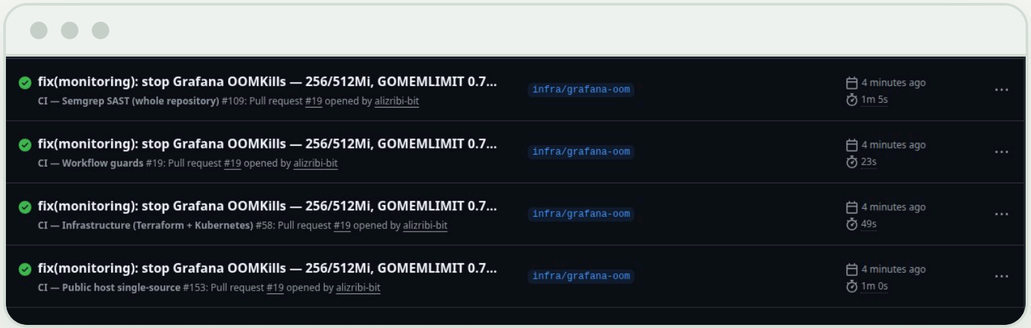
rien n'est publié

fusion →

Push sur staging, après fusion

Si un composant a changé :

- image publiée dans ACR
- manifeste mis à jour



06/10 · PR #19 (configuration du cluster) : contrôles verts, aucune image

Mobile

APK testé et construit ; sur push, envoyé dans le stockage Azure et par e-mail. Pas de pod.

Branche main

Même publication configurée ; la production n'est pas déployée.

Notre chaîne CI/CD · Tests

Ce que les tests vérifient

API

`JUnit`

- score d'achat local
- page publique du certificat
- limite des tentatives de connexion
- en-têtes de sécurité

≈ 180 tests

Site client

`Vitest`

- demandes de devis
- historique carbone
- accès bloqué sans grille validée

≈ 760 tests

Service d'IA

`pytest`

- contrat avec l'API Java
- calcul carbone
- rapport ESG sans chiffre inventé

27 tests

Mobile

`flutter test`

- score de la grille
- palier de certification
- marketplace

≈ 540 tests

Un test en échec arrête le pipeline : l'image n'est pas construite, rien n'est publié.

L'administration n'a encore que quelques tests : c'est un point à compléter.

Notre chaîne CI/CD · Analyses de sécurité

Cinq familles d'analyses

Ce qu'on analyse	Outil	Où il tourne	Effet
Secrets	`Gitleaks`	chaque pipeline de composant, et tout le dépôt	bloque
Code	`SonarCloud` `Semgrep`	API Java · tout le dépôt (JavaScript, React, Python)	PR et main rapport
Bibliothèques	`Dependency-Check`	dépendances Java de l'API	bloque
Images	`Trivy`	images de l'API, de l'IA et des deux sites	bloque
Infrastructure	`Checkov`	fichiers Terraform et Kubernetes	rapport

Chaque pipeline n'a pas toutes les analyses : il lance celles qui concernent ses fichiers. Semgrep produit un rapport ; SonarCloud attend sa quality gate sur les pull requests et sur main.

Notre chaîne CI/CD · Contrôles

Bloquer, signaler, protéger la fusion

Arrête le pipeline

le job échoue, rien n'est publié

`Gitleaks`
un secret est trouvé

`Dependency-Check`
faille de score CVSS 9 ou plus

`Trivy`
faille HIGH ou CRITICAL qui a un correctif ; 2 exceptions datées

`SonarCloud`
quality gate, sur PR et main seulement

Signale seulement

un rapport à lire, le job continue

`Semgrep`
les résultats ne font pas échouer le job

`Checkov`
analyse marquée non bloquante

Protège la fusion

rôle de GitHub, pas des outils

`Protection de branche`
non disponible : dépôt privé, offre gratuite

`Conséquence`
un pipeline rouge n'empêche pas de fusionner

`Règle d'équipe`
on ne fusionne pas tant que c'est rouge

Notre chaîne CI/CD · Essai réel

Un contrôle bloquant mis à l'épreuve



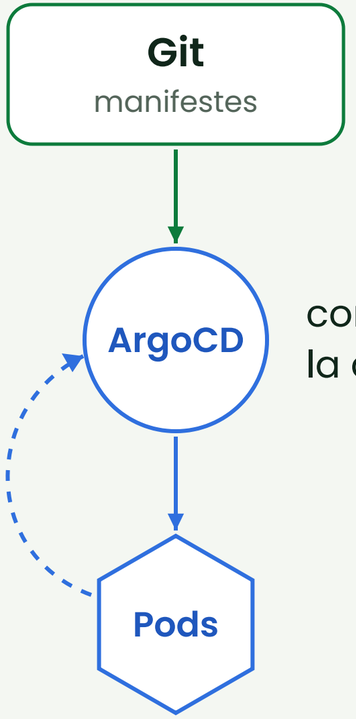
Bibliothèque retirée : le pipeline redevient vert

Le rouge venait de log4j.

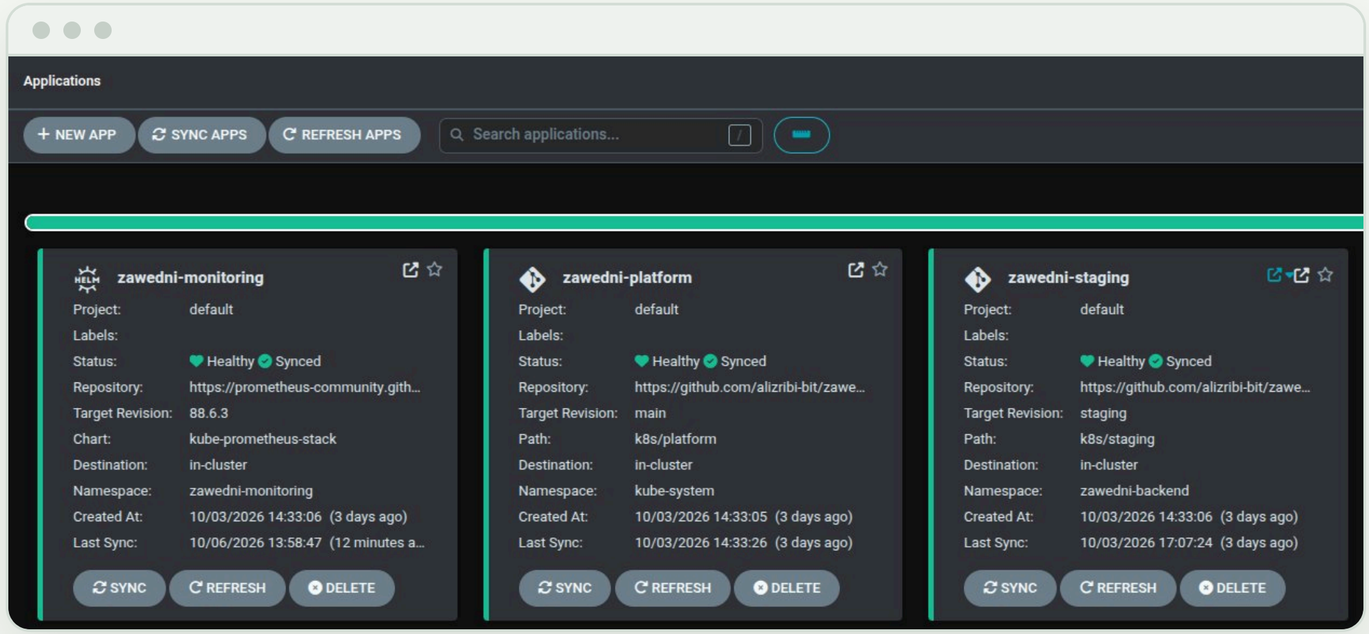
Essais volontaires : log4j · faux secret · ancienne adresse

Notre chaîne CI/CD · GitOps

Déploiement GitOps avec ArgoCD



compare et applique la différence



Capture du 06/10 · zawedni-platform, zawedni-monitoring, zawedni-staging : Synced, Healthy

Mises à jour de l'application : le pipeline change le manifeste, ArgoCD applique.

Traçabilité d'un changement

1 run · 62c42e1 → 2 ACR · 62c42e1 → 3 manifeste · 62c42e1 → 4 pod · 62c42e1

Cluster AKS · staging · zawedni-backend

```
IMAGE                                                                                                READY
zawedniacr.azurecr.io/zawedni-backend:62c42e1d02f81db4725b3e35dc57fa1d7c2c1c0c-staging               true
```

62c42e1 Pod staging · Ready · 06/10

Chapitre 5

05

Supervision et validation

Voir ce qui se passe, et prouver que cela fonctionne.

01 Contexte et objectifs

02 Développement de la plateforme

03 Architecture et DevSecOps

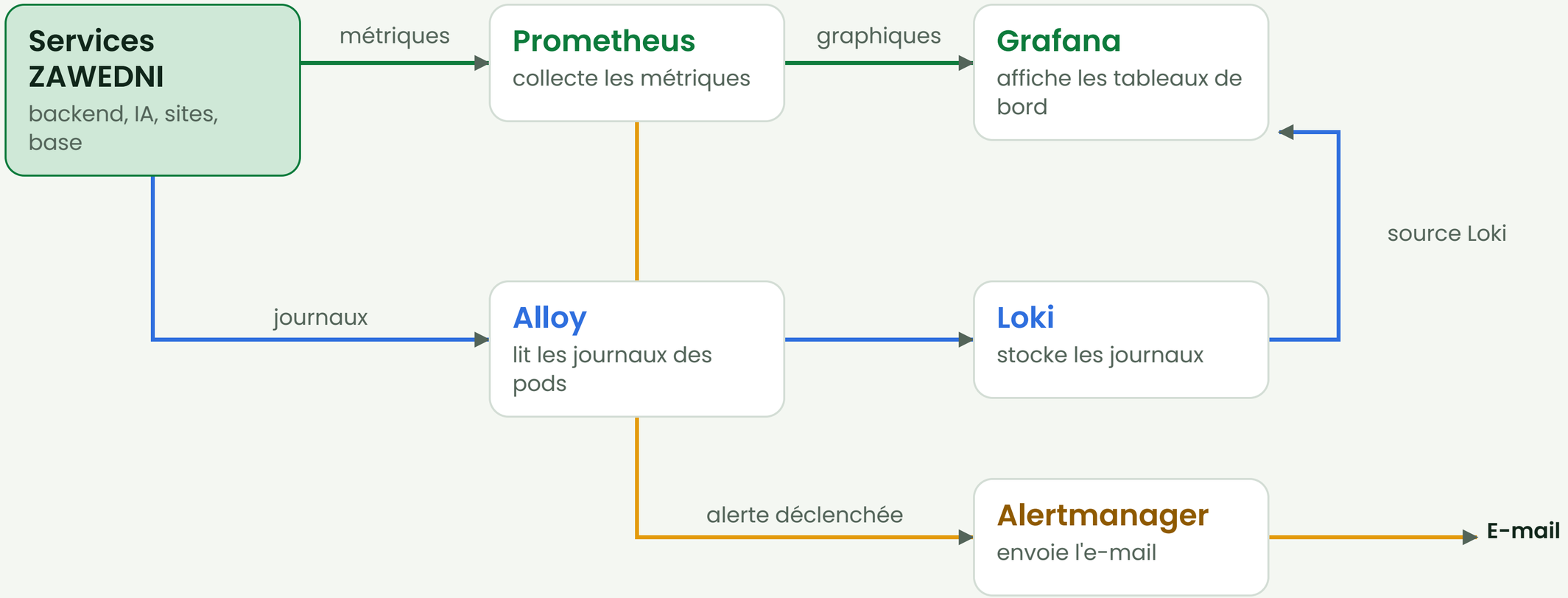
04 Chaîne CI/CD et déploiement

05 Supervision et validation

06 Bilan et perspectives

Supervision et validation · Rôles

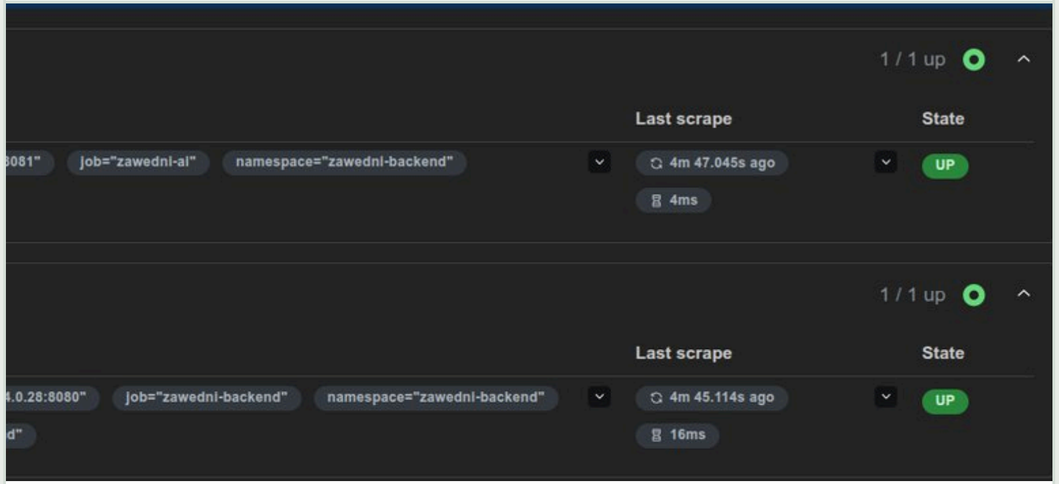
Quatre outils de supervision



Supervision et validation · Métriques

Métriques des services

Prometheus · cibles



06/10 · le backend et le service IA exposent leurs métriques : cibles UP

Grafana · ressources des pods



06/10 · CPU et mémoire des pods backend et IA, dernière heure

Journaux centralisés

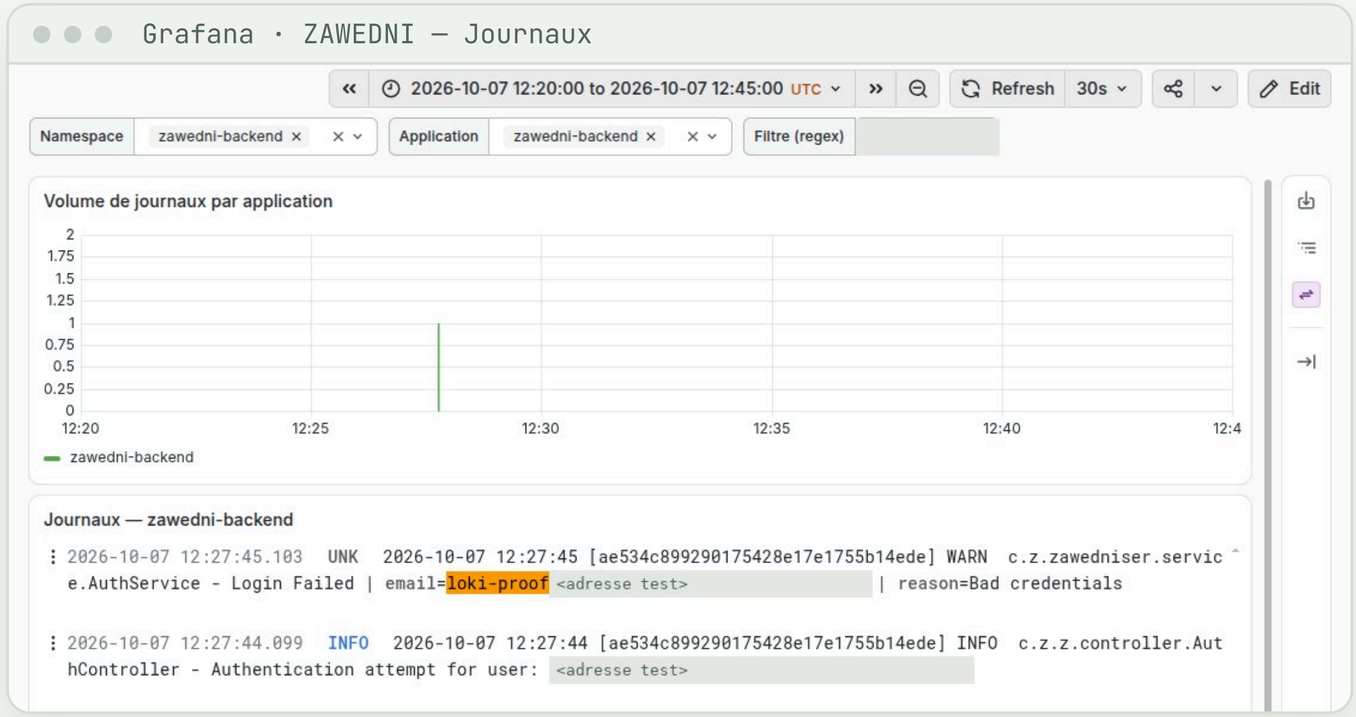
1 · Action réelle
connexion échouée sur staging (401)

2 · Backend
écrit deux lignes de journal

3 · Alloy
les collecte dans le pod

4 · Loki
les stocke, avec leurs étiquettes

5 · Grafana
les affiche et les filtre



07/10, 12:27 UTC
Les deux lignes arrivent dans Loki une seconde après la requête.

Accès limité
Grafana et Alloy joignent Loki ; un pod non autorisé est bloqué.

Supervision et validation · Alertes

Une alerte, de bout en bout

Erreurs 500
12:29 · 136 requêtes

En attente
12:31 · seuil dépassé

Déclenchée
12:33 · Alertmanager

E-mail FIRING
12:33 · reçu

Résolue
12:39

E-mail RESOLVED
12:38 · reçu

12:34 · charge arrêtée

Prometheus · règle ZawedniHighErrorRate déclenchée

ZawedniHighErrorRate

FIRING (1)

```
rate(http_server_requests_seconds_count{job="zawedni-backend",namespace="zawedni-backend",status=~"5.."}[5m]) > 0.1
```

for: 2m

Gmail · heure locale (UTC+1)

[RESOLVED] ZawedniHighErrorRate (zawedni-backend http none none 10.244.0 1:38 PM

[FIRING:1] ZawedniHighErrorRate (zawedni-backend http none none 10.24 1:33 PM

Portée de l'essai

Cinq règles chargées et saines. Une seule, le taux d'erreurs, a été déclenchée.

Staging, 07/10 (UTC) · erreurs provoquées par une requête en lecture seule, arrêtées après cinq minutes

Supervision et validation · Résultats

Ce qui a été validé

Vérification	Ce qu'elle démontre	Résultat
Parcours web et mobile	écrans web parcourus sur la version locale ; écrans mobiles couverts par leurs tests	validé en local
Accès aux services	staging répond : site, administration et API (200)	validé
État GitOps	les applications ArgoCD sont synchronisées et saines	validé
Contrôles bloquants	un secret ou une bibliothèque vulnérable ajoutés exprès arrêtent le pipeline	validé
Isolation réseau	10 tests sur 11 ; la résolution DNS reste à clarifier	partiel
Supervision	journal collecté ; une alerte déclenchée puis résolue	validé

Non validé ici : la production, l'arrêt puis le redémarrage du cluster, la tenue sur plusieurs jours

Chapitre 6

06

Bilan et perspectives

Ce qui est réalisé, ses limites et la suite.

01 Contexte et objectifs

02 Développement de la plateforme

03 Architecture et DevSecOps

04 Chaîne CI/CD et déploiement

05 Supervision et validation

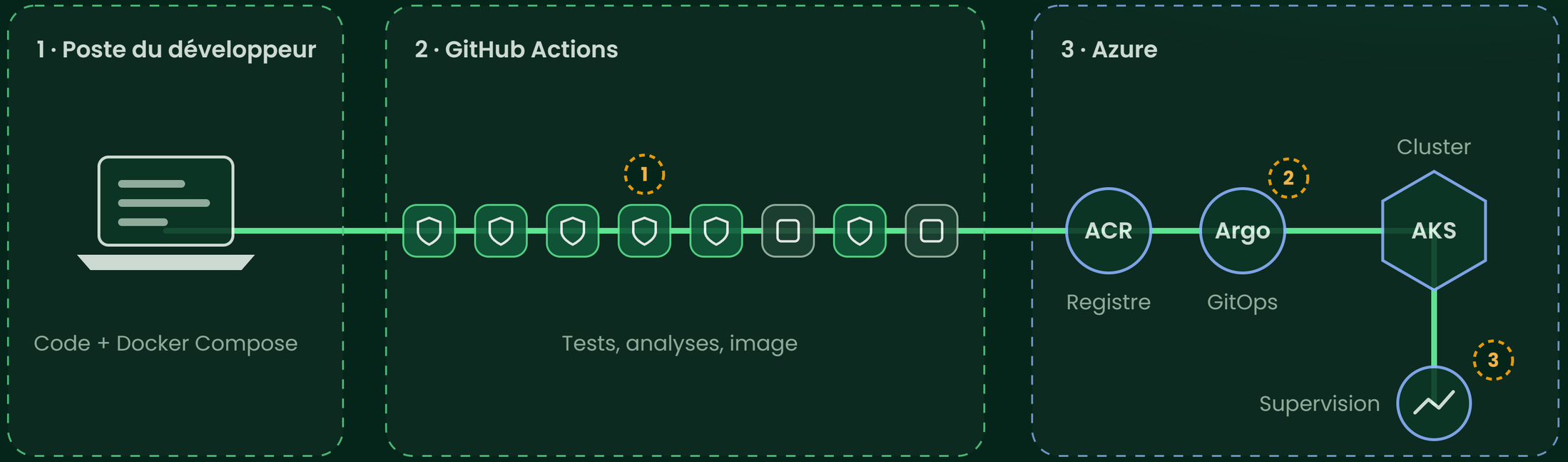
06 Bilan et perspectives

Bilan du projet

Volet	Moyens	Résultat	État
Plateforme	web, mobile, API, IA	évaluation RSE, certificat, marketplace, devis, carbone, ESG	Réalisé
Infrastructure	Terraform, AKS, ACR	cluster recréé depuis le code	Réalisé
Livraison	10 pipelines, GitOps	changement publié en staging, suivi jusqu'au pod	Staging : réalisé Production : configurée
Sécurité	5 familles d'analyses	secrets, bibliothèques, images bloquent ; secrets chiffrés	Réalisé Fusion non protégée
Supervision	métriques, journaux, alertes	une alerte déclenchée, reçue par e-mail, résolue	Réalisé en staging

Perspectives

1 **Sécurité** HTTPS, protection des branches, Semgrep et Checkov bloquants



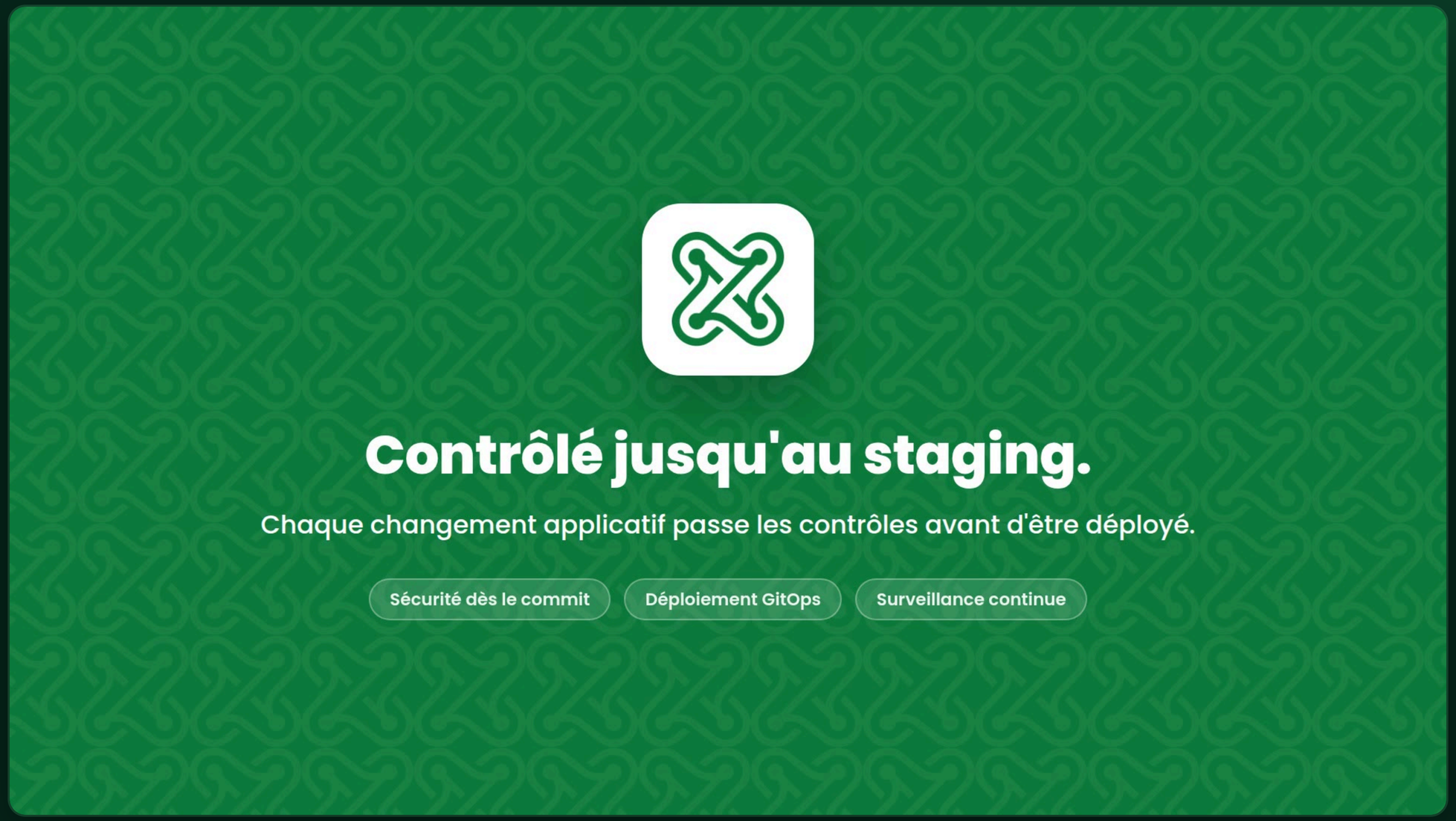
2 **Exploitation** production et sauvegardes

3 **Supervision** alertes en production, base surveillée

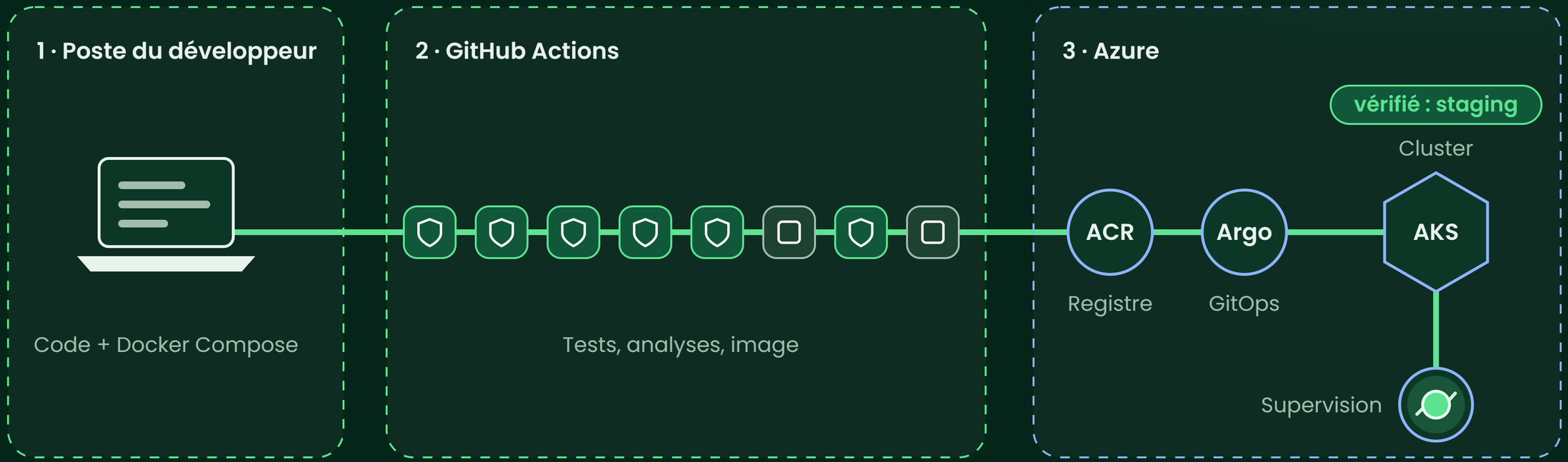
Déjà réalisé et retiré de la liste : journaux centralisés, alertes par e-mail en staging.

Bilan et perspectives · Récapitulatif

Le voyage d'un changement, en une minute trente



Conclusion



Merci pour votre attention